

Chapter 1

Conclusion and Future Work

This final chapter closes the thesis. Section 1.1 summarises the problems the project set out to solve and what it achieved, and then states honestly the limitations that remain. Section 1.2 builds on those limitations to propose concrete directions for future development.

1.1 Summary

The objective of this project was to build a web intrusion detection system that overcomes the central weakness of the existing literature surveyed in Chapter 2, namely the reliance on a single detection paradigm. Signature-based detectors are precise but blind to new attacks, purely supervised detectors require labelled data and cannot generalise beyond the attack families they were trained on, and purely unsupervised detectors catch novel anomalies but at the price of many false alarms. The project addressed this gap by designing, implementing, and evaluating a three-tier hybrid system that fuses all three paradigms, and the work delivered the following concrete results.

- (i) **A working three-tier hybrid detector.** The system combines a signature- and rule-based tier, a supervised Random Forest tier, and an unsupervised one-class anomaly tier, and reconciles their opinions through a smart consensus rule in which the supervised model validates the anomaly tier. This design, detailed in Chapter 3, is the core contribution of the thesis.
- (ii) **A balanced benchmark dataset.** A corpus of 111,065 log lines was assembled from the CSIC 2010 dataset, real attack payloads from a public repository, and synthetic traffic, balanced evenly between attack and clean requests, and partitioned so that the one-class tier could be trained on clean traffic alone.
- (iii) **Operation on raw server logs.** Unlike most prior work, which operates on pre-processed feature sets, the system works directly on Apache access logs. It performs multi-layer URL decoding to defeat encoding-based evasion, supports attacks delivered through the POST body, and represents every request with a compact, shared 22-feature vector.
- (iv) **Strong, recall-oriented results.** As reported in Chapter 4, the supervised tier alone reaches an F1 of 94.78% and the one-class tier alone an F1 of 91.57%, while the proposed smart consensus achieves the system’s best F2 score of 96.26%, combining a recall of 97.62% with a precision of 91.19%. The consensus therefore meets the

operational priority of an intrusion detection system, which is to minimise missed attacks.

- (v) **Explainability and a usable tool.** Because the verdict is a transparent combination of three named signals, every alert is annotated with the tiers that produced it, and the system ships with a real-time monitoring mode and a dashboard, making it usable in practice rather than only as an offline experiment.

Despite these results, several limitations remain and should be acknowledged. First, the models were trained on the CSIC 2010 e-commerce traffic together with generic web traffic, so when deployed on an application with very different normal behaviour (for example a content-management system or a bespoke API) the system may raise more false positives until it is retrained on that application’s own clean logs. Second, the unsupervised tier assumes a stable notion of “normal” traffic; if the legitimate behaviour of the protected application drifts over time, previously unseen but benign patterns may be flagged as anomalies, a problem only partially mitigated by the smart consensus. Third, the anti-evasion decoding handles a bounded number of encoding layers, so payloads hidden behind an unusually deep or binary encoding can still slip through. Fourth, the system inspects textual request content, so attacks carried in binary POST bodies are not analysed. Finally, the evaluation, although conducted on a sizeable balanced set, was performed on a single dataset family and on logs already decrypted by the server; the system performs detection and alerting only and does not, by itself, block malicious requests. These limitations directly motivate the directions proposed next.

1.2 Suggestion for Future Works

Building on the limitations identified above, several avenues for future development are worth pursuing.

- (i) **Evaluation on more and more realistic datasets.** The most important next step, also recommended by Pham et al. [?], is to validate the system on additional datasets, especially traffic collected from a real production environment, so that its generalisation beyond CSIC 2010 can be measured rather than assumed.
- (ii) **Semi-supervised and online learning.** Following the suggestion of Althubiti et al. [?], semi-supervised techniques could reduce the labelling cost while retaining accuracy. In parallel, an online or incremental learning scheme that periodically refreshes the models on recent clean traffic would address the baseline-drift limitation of the unsupervised tier.
- (iii) **Concept-drift detection and an analyst feedback loop.** The system could monitor its own false-positive rate and trigger retraining when drift is detected, and it could incorporate the analyst’s confirmations and dismissals of alerts as new labels, turning day-to-day operation into a continuous improvement process.
- (iv) **Richer representations for the anomaly tier.** Replacing or augmenting the hand-crafted feature vector with a learned representation, for instance from an autoencoder or a sequence model over the request text, could let the unsupervised tier capture semantic anomalies that the current features miss, while preserving the one-class training discipline.

- (v) **Stronger evasion and adversarial resistance.** Extending the decoder to handle deeper and binary encodings, and explicitly testing the system against adversarially crafted payloads, would harden it against attackers who deliberately target the detector.
- (vi) **From detection to prevention.** The system could be integrated with a web application firewall so that high-confidence alerts trigger automatic blocking, turning the present detection-only pipeline into an active prevention mechanism, with the consensus score serving as a natural, tunable threshold for the blocking decision.
- (vii) **Production deployment at scale.** In a real deployment the detector would operate inside a streaming pipeline rather than process a static log file, and it would be combined with several standard infrastructure components. Web servers (Apache or Nginx) would ship their access logs through a lightweight forwarder such as Filebeat or Fluent Bit into an *Apache Kafka* topic that acts as a buffering queue; this decouples the rate at which logs are produced from the rate at which they are analysed and absorbs sudden traffic spikes without losing events. The IDS itself would be packaged as a container and run as one or more Kafka consumers on a *Kubernetes* cluster: when Kafka reports that the queue is growing (a traffic surge), a horizontal autoscaler—for example KEDA driven by the Kafka consumer lag—would automatically replicate the IDS from a single container to many parallel ones, and scale them back down once the backlog drains, so the system neither loses data nor wastes resources. The JSON alerts emitted by the consensus stage would then be forwarded to a *SIEM* platform (such as Elastic/ELK, Splunk, Wazuh, or IBM QRadar) for correlation with other security events, long-term storage, dashboards, and incident response; the highest-confidence alerts could additionally drive the web application firewall of the previous point to block offending requests in real time. Designing, implementing, and benchmarking this architecture under production load is a natural direction for future engineering work.

Taken together, these directions would extend the thesis from a strong, explainable, recall-oriented detector validated on a benchmark dataset towards an adaptive, self-improving, and preventive system suitable for long-term deployment in a changing production environment.